Java Debugging & Performance — Interview Q&A;

20 common production troubleshooting questions, explained simply

1. Your Java application suddenly starts throwing OutOfMemoryError. How will you debug it?

Take a heap dump when it crashes and open it in a tool like Eclipse MAT. See which objects are eating memory. Usually it's a leak — objects that should get garbage collected but don't (like a list that keeps growing, or a cache with no limit).

2. You see high CPU usage but low traffic. What could be the reason?

Something is looping badly. Take a thread dump, find which thread is using the most CPU, and see what code it's stuck in. Common causes: infinite loop, bad regex, or a job running in a loop by mistake.

3. A thread is stuck in BLOCKED state. How will you identify and fix it?

Take a thread dump. It shows which lock the thread is waiting for, and which thread is holding that lock. Usually one thread is holding a lock too long (doing slow work inside synchronized). Fix: make the locked section smaller, or use better locking (like ReentrantLock).

4. Your application slows down after running for a few hours. What will you check?

This smells like a memory leak or resource leak. Check if memory usage keeps going up over time, if too many threads are open, or if DB connections aren't closing properly.

5. You are facing frequent GC pauses. How will you optimize it?

Look at GC logs to see what's happening. Usually the app is creating too many objects too fast. Fix by reducing unnecessary object creation, tuning heap size, or switching to a better garbage collector (like G1 or ZGC).

6. A HashMap is causing performance issues under heavy load. Why?

HashMap is not thread-safe. If many threads use it at once, it can break or behave weirdly. Fix: use ConcurrentHashMap instead.

7. Multiple threads are updating shared data incorrectly. How will you fix it?

This is a race condition — two threads changing the same data at the same time. Fix: use synchronized, ReentrantLock, or thread-safe classes like AtomicInteger and ConcurrentHashMap.

8. Your API works fine locally but fails in production. What will you investigate?

Compare the two environments. Check config files, environment variables, JVM version, DB connection, and data size. Production often has more data or different settings than your local setup.

9. You suspect a memory leak. How will you confirm it?

Take two heap dumps at different times, compare them. If some objects keep increasing and never go away, that's your leak.

10. A service becomes unresponsive randomly. What could be happening?

Could be a long GC pause, thread pool full, deadlock, or a slow external call with no timeout. Take a thread dump exactly when it freezes to see what's stuck.

11. You see deadlock in your system. How will you detect and resolve it?

jstack (thread dump) directly tells you if there's a deadlock and shows which threads/locks are involved. Fix: always lock things in the same order everywhere in your code, and use lock timeouts instead of waiting forever.

12. Your logs show inconsistent behavior across requests. Why?

Usually caused by shared state (a variable that many requests use together) or race conditions. Also check for cache issues or code that behaves differently based on time/order.

13. Your application crashes without any clear error. How will you debug it?
Check for a native crash log file, or check if the OS killed the process for using too much memory (common in Docker/Kubernetes — check dmesg or pod logs).

14. A database call is slowing down your Java service. How will you optimize it?
Check if the query has an index, avoid running many small queries when one big one would do (N+1 problem), and check your connection pool size.

15. Your thread pool gets exhausted under load. What's your approach?
All your worker threads are busy or stuck. Fix by adding timeouts to slow calls, sizing your thread pool properly, and using backpressure so you don't accept more work than you can handle.

16. You need to handle high concurrency safely. What will you use?
Use Java's built-in concurrency tools: ConcurrentHashMap, ExecutorService (thread pools), AtomicInteger, CompletableFuture. Avoid shared mutable state when possible.

17. Your system processes duplicate requests. How will you handle it?
Use an idempotency key — a unique ID per request. Before processing, check if that ID was already handled (store it in DB/cache). If yes, skip or return the same result.

18. A cache is giving stale data. How will you fix it?
Set a proper expiry (TTL) on cache entries, and invalidate/update the cache whenever the underlying data changes (write-through or cache-eviction pattern).

19. Your application is not scaling even after adding instances. Why?
Usually a bottleneck outside the app — like a single database, a shared lock, or a session stored only on one server. Check what all instances depend on that isn't scaled too.

20. You need to trace a request across multiple layers. How will you do it?
Use a correlation ID / trace ID — generate one when the request comes in, and pass it through every layer and log line. Tools like Zipkin, Jaeger, or Spring Cloud Sleuth help visualize the full path.

Cheat Sheet — Fast Reference

All 20 answers at a glance for quick revision

#	Scenario	Quick Answer
1	**OutOfMemoryError**	Take a heap dump on crash, analyze in Eclipse MAT/VisualVM. Find growing objects, unbounded lists/caches.
2	**High CPU, low traffic**	Thread dump + `top -H` to find the hot thread. Look for infinite loops, bad regex, bad job loops.
3	**Thread stuck BLOCKED**	Thread dump shows lock owner. Usually a long `synchronized` block. Shrink critical section or use ReentrantLock.
4	**Slows down after hours**	Likely memory/resource leak. Watch heap trend, thread count, and DB connection pool over time.
5	**Frequent GC pauses**	Check GC logs. Reduce object churn, tune heap size, try G1/ZGC collector.
6	**HashMap under heavy load**	HashMap isn't thread-safe; concurrent access can corrupt it. Use ConcurrentHashMap.
7	**Threads corrupting shared data**	Classic race condition. Use synchronized/ReentrantLock, or Atomic*/ConcurrentHashMap.
8	**Works locally, fails in prod**	Compare configs, JVM version, env vars, DB, data volume, and resource limits.
9	**Confirm memory leak**	Take two heap dumps over time and diff them. Objects that keep growing = leak.
10	**Randomly unresponsive**	Long GC pause, thread pool exhaustion, deadlock, or slow call with no timeout. Grab a thread dump at the moment.
11	**Deadlock detected**	jstack flags deadlocks directly. Fix: consistent lock ordering + tryLock() with timeout.
12	**Inconsistent behavior in logs**	Shared mutable state or race condition. Also check caching and iteration order issues.
13	**Crash, no clear error**	Check hs_err_pid log (native crash) or OS OOM-killer (check dmesg / container logs).
14	**Slow DB call**	Check query plan/indexes, avoid N+1 queries, review connection pool size.
15	**Thread pool exhausted**	Add timeouts to slow calls, size pool correctly, use backpressure/bounded queues.
16	**Safe high concurrency**	Use java.util.concurrent: ConcurrentHashMap, ExecutorService, Atomic*, CompletableFuture.
17	**Duplicate requests**	Use an idempotency key; check/store it before processing to skip repeats.
18	**Cache giving stale data**	Set TTL/expiry, and invalidate or refresh cache on data change (write-through).
19	**Not scaling with instances**	Find the shared bottleneck: single DB, shared lock, or session stuck on one server.
20	**Trace request across layers**	Use a correlation/trace ID through every layer and log line; tools: Zipkin, Jaeger, Sleuth.